

Universidad Católica de Temuco

Ingeniería Civil en Informática

# INFORME DE TAREA

## Análisis léxico del lenguaje Prolog

*Teoría de la Computación*

| Dato             | Información                                     |
|------------------|-------------------------------------------------|
| Código curso     | INFO1148                                        |
| Integrantes      | Fernando Valdés<br>Juan Muñoz<br>Vicente Rivera |
| Profesor         | Prof. M. Lévano                                 |
| Fecha de entrega | 21 de septiembre de 2026                        |

*Temuco, 2026*

## Resumen

Este informe presenta el análisis, diseño, implementación y validación de un analizador léxico para un subconjunto de Prolog inspirado en la notación de ISO Prolog y SWI-Prolog. El trabajo define formalmente el catálogo de tokens con una expresión regular por categoría, modela las categorías principales con autómatas finitos, muestra la determinización por construcción de subconjuntos y la minimización por refinamiento de particiones sobre un subconjunto representativo, e implementa el lexer en Python con recorrido de izquierda a derecha, máxima coincidencia y prioridades explícitas. El lexer emite tipo, lexema original, línea y columna por token, administra una tabla de lexemas con deduplicación por par tipo y lexema, y reporta seis clases de error con fragmento y posición continuando el análisis. La validación usa una suite de 85 pruebas automatizadas con 26 casos válidos, 22 de prioridad, 16 inválidos y dos archivos completos, con resultado real de 85 aprobadas y cero fallos.

**Palabras clave:** análisis léxico, Prolog, expresiones regulares, autómatas finitos, tabla de lexemas.

## Introducción

El análisis léxico es la primera fase de un compilador: transforma la cadena fuente en una secuencia ordenada de tokens con posición, separando las decisiones regulares (qué formas son válidas) de las decisiones sintácticas (en qué orden pueden aparecer). Un lexer correcto y bien diagnosticado simplifica el parser, produce errores comprensibles y deja evidencia reproducible de cada decisión.

El trabajo se plantea como un recorrido de izquierda a derecha. En cada posición el lexer identifica la categoría que puede comenzar allí, busca el lexema válido más largo y aplica la prioridad definida cuando dos categorías empatan. Este orden conecta las expresiones regulares, los autómatas y la implementación: los diagramas explican los recorridos posibles, mientras que el código decide qué recorrido tiene precedencia cuando las formas se superponen.

El documento avanza desde el problema hacia la evidencia. Primero fija el alcance y el catálogo; después formaliza las expresiones regulares y los autómatas; a continuación describe la arquitectura, la tabla de lexemas y la recuperación de errores; finalmente presenta las pruebas reproducibles y las conclusiones. La separación entre especificación, implementación y validación permite localizar una regla en la especificación, seguirla en el código y comprobarla en una prueba.

## Objetivos

### Objetivo general

Analizar, diseñar, implementar y validar un analizador léxico para un subconjunto definido de Prolog, aplicando alfabetos, lenguajes regulares, expresiones regulares y autómatas finitos, con pruebas reproducibles y diagnósticos verificables.

### Objetivos específicos

- Definir cada categoría con nombre de token, patrón, ejemplos válidos e inválidos y atributo asociado.
- Construir una expresión regular por categoría y resolver superposiciones con máxima coincidencia y prioridades explícitas.
- Modelar las categorías principales con AFN o AFD e identificar estados iniciales, de aceptación y de error.
- Mostrar la determinización por construcción de subconjuntos y la minimización con justificación de equivalencias sobre un subconjunto representativo.
- Implementar el lexer con posiciones 1-based, tabla de lexemas y seis clases de error recuperables.

- Validar con al menos 20 pruebas válidas, 8 inválidas, casos de prioridad y dos archivos completos.

## Alcance del analizador

El analizador reconoce exactamente el catálogo de `docs/informe/01_especificacion_lexica.md`: átomos simples y entre comillas simples, variables nombradas y anónima, enteros y reales sin signo, cadenas con comillas dobles, operadores de las seis familias (cláusula, consulta, DCG, unificación y comparación, aritméticos y control), nueve delimitadores de un carácter y elementos ignorables (espacios, tabulaciones, saltos de línea y comentarios de línea y de bloque).

Quedan fuera del alcance el análisis sintáctico y semántico: no se verifica el orden de los tokens, no se construye un árbol sintáctico, no se comprueba el balance de paréntesis, corchetes o llaves, no se resuelven ámbitos ni variables repetidas, no se implementa unificación, no se evalúan expresiones y no se decide si una secuencia de tokens forma un programa válido. Una `)` aislada es un token `RPAREN` correcto; su posible improcedencia pertenece a una fase posterior.

## Especificación léxica

| Token              | Descripción                                                                      | Expresión regular              | Ejemplos válidos                          |
|--------------------|----------------------------------------------------------------------------------|--------------------------------|-------------------------------------------|
| ATOM               | Átomo iniciado por minúscula ASCII, seguido de minúsculas, dígitos o guion bajo. | <code>[a-z][a-z0-9_]*</code>   | <code>padre, persona_1, isla</code>       |
| QUOTED_ATOM        | Átomo entre comillas simples, con escapes válidos y sin salto real.              | <code>'...' con escapes</code> | <code>'Juan Perez', ':-', 'it\'s'</code>  |
| VARIABLE           | Mayúscula inicial o guion bajo seguido de al menos un carácter identificador.    | <code>[A-Z]...   _...</code>   | <code>X, Persona, _Tmp</code>             |
| ANONYMOUS_VARIABLE | Guion bajo aislado, sin continuación.                                            | <code>_</code>                 | <code>_</code>                            |
| INTEGER            | Dígitos ASCII sin signo.                                                         | <code>[0-9]+</code>            | <code>0, 25, 0007</code>                  |
| REAL               | Dígitos, punto y dígitos obligatorios a ambos lados.                             | <code>[0-9]+\.[0-9]+</code>    | <code>3.14, 0.5, 10.00</code>             |
| STRING             | Literal entre comillas dobles, con escapes válidos y sin salto real.             | <code>"..." con escapes</code> | <code>"hola", "dice"</code>               |
| OPERATOR           | Operador exacto de seis familias, por longitud descendente.                      | unión normativa (secc. 6)      | <code>:-, ?-, -&gt;, =. ., //, mod</code> |
| Delimitadores      | Nueve símbolos de un carácter, sin validación de balance.                        | un símbolo                     | <code>( ) [ ] { }   , .</code>            |

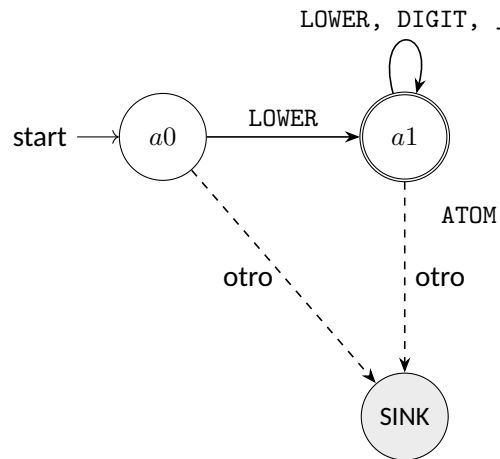
Las familias de `OPERATOR` son cláusula (`:-`), consulta (`?-`), DCG (`->`), unificación y comparación (`=`, `\=`, `==`, `\==`, `=. .`, `<`, `=<`, `>`, `>=`), aritméticos (`+`, `-`, `*`, `/`, `//`, `**`, `is`, `mod`) y control (`\+`, `!`, `;`). Las palabras `is` y `mod` se reconocen solo cuando el lexema completo no tiene continuación de identificador.

Los números no tienen signo; `-12` son dos tokens. `INTEGER` y `REAL` no admiten exponentes ni puntos sin dígitos a ambos lados. Los escapes válidos son la barra con el delimitador, más `\n`, `\r` y `\t`. Ningún literal

admite un salto de línea real. El signo % inicia un comentario de línea hasta antes del salto y /\* abre un bloque que termina en el primer \*/ sin anidamiento. Las posiciones son 1-based; la tabulación avanza una columna y CRLF cuenta como un único salto. El lexema conservado es siempre el segmento original, sin normalización ni conversión.

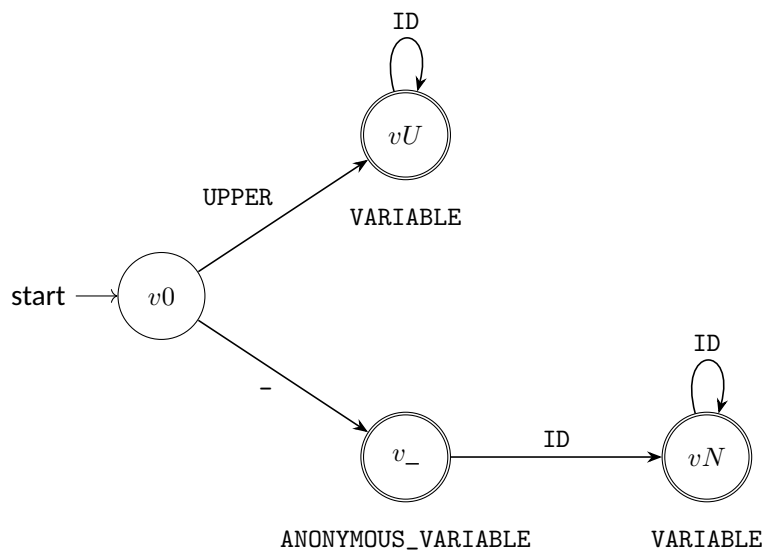
## Diseño de los autómatas

Las figuras usan la convención estado –símbolo–> estado; el doble círculo indica aceptación y el rótulo inferior el token emitido. Los alfabetos abreviados son LOWER=[a-z], UPPER=[A-Z], DIGIT=[0-9] e ID=[A-Za-z0-9\_].



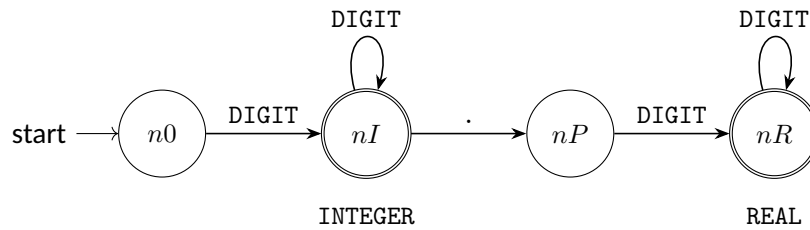
**Figura 1:** Autómata para átomos no entrecomillados.

El AFD de átomos reconoce LOWER (LOWER U DIGIT U {\_})\* con máxima coincidencia:  $q_0 \xrightarrow{\text{LOWER}} q_1$  y  $q_1 \xrightarrow{[a-z0-9_]} q_1$ , donde  $q_1$  acepta ATOM.



**Figura 2:** Autómata para variables y variable anónima.

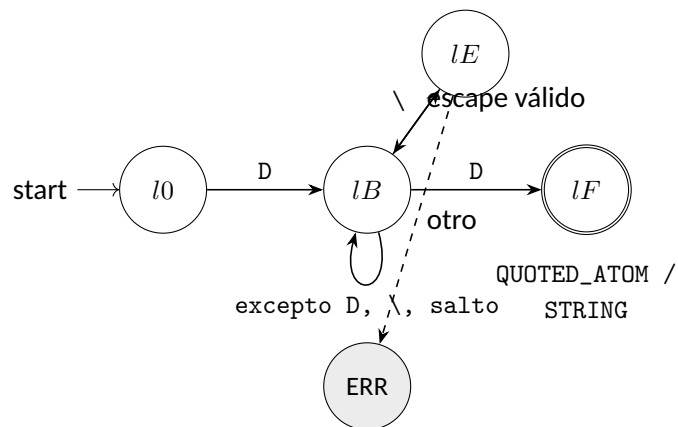
El AFD de variables distingue la variable con nombre de la anónima:  $v_U$  y  $v_N$  aceptan VARIABLE, mientras que  $v_-$  sin continuación se resuelve como ANONYMOUS\_VARIABLE por regla de longitud y prioridad.



**Figura 3:** Autómata para números enteros y reales.

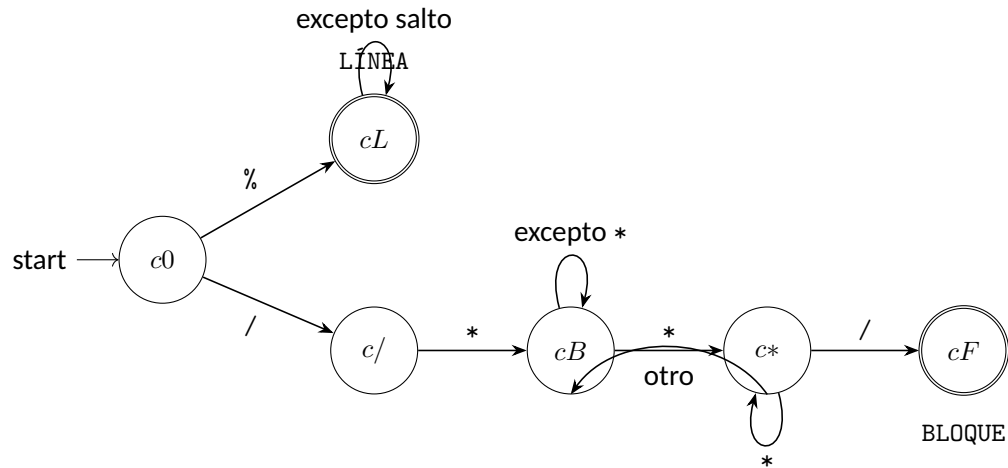
El AFD de números acepta en  $nI$  (INTEGER) y  $nR$  (REAL);  $nP$  no acepta, por lo que `5.` no pertenece al lenguaje numérico y se diagnostica como número mal formado.

Los literales entrecomillados siguen el esquema `D . . . D` con estado de escape: el delimitador abre el literal, los caracteres seguros ciclan, la barra lleva al estado de escape y solo un escape válido o el delimitador de cierre permiten continuar. Un salto de línea real o un escape desconocido no tienen transición válida y producen el error correspondiente.



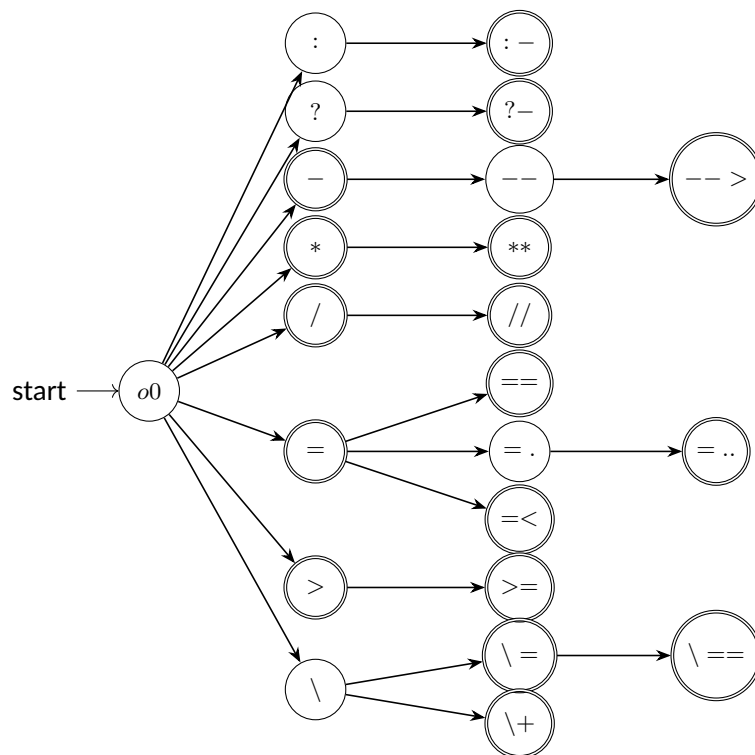
**Figura 4:** Autómata para literales entrecomillados (átomos citados y cadenas).

Los comentarios separan la rama de línea (`%` hasta el salto, que acepta de inmediato y cicla hasta el fin de línea) de la rama de bloque (`/* . . . */` hasta el primer cierre). Solo la secuencia `*/` cierra la rama de bloque; cualquier otro símbolo, incluido un `*` aislado, mantiene el recorrido dentro del comentario.



**Figura 5:** Autómata para comentarios de línea y de bloque.

Los operadores se organizan en un trie que comparte prefijos para aplicar máxima coincidencia antes de emitir el token: cada camino desde `o0` deletrea el lexema y solo los nodos de doble círculo aceptan, conservando lexema y familia. Los operadores de un carácter ( , ; ! + - \* / < = > ) aceptan directamente desde `o0`; las palabras `is` y `mod` requieren además la guarda de frontera de palabra.



**Figura 6:** Trie de operadores: prefijos compartidos y aceptación por máxima coincidencia.

El despachador integrado decide por la primera clase de carácter y luego aplica máxima coincidencia y prioridad, lo que equivale a unir los autómatas por transiciones epsilon, determinizar y etiquetar cada aceptación.

Para mostrar el procedimiento completo se tomó el lenguaje representativo  $\{=, ==, \backslash=, \backslash==\}$ . El AFN une

mediante epsilon las ramas  $q0 \rightarrow q1 = q2 = q3$  y  $q0 \rightarrow q4 \setminus q5 = q6 = q7$ ; los estados  $q2, q3, q6$  y  $q7$  aceptan  $OP\_EQ, OP\_EQQ, OP\_NEQ$  y  $OP\_NEQQ$  con etiquetas distintas porque el lexer debe producir lexemas distintos.

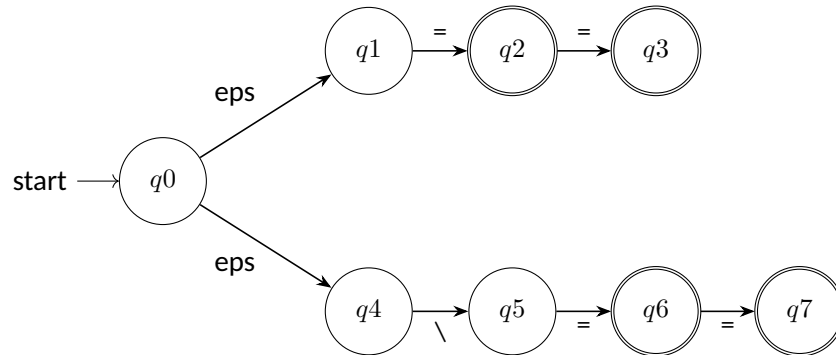


Figura 7: AFN representativo de operadores.

La construcción de subconjuntos comienza con A como clausura epsilon de  $\{q0\}=\{q0, q1, q4\}$ . La tabla contiene todos los subconjuntos alcanzables, incluido el estado muerto Z.

| Estado | Subconjunto AFN  | Con = | Con \ | Aceptación |
|--------|------------------|-------|-------|------------|
| A      | $\{q0, q1, q4\}$ | B     | D     | ninguna    |
| B      | $\{q2\}$         | C     | Z     | OP_EQ      |
| C      | $\{q3\}$         | Z     | Z     | OP_EQQ     |
| D      | $\{q5\}$         | E     | Z     | ninguna    |
| E      | $\{q6\}$         | F     | Z     | OP_NEQ     |
| F      | $\{q7\}$         | Z     | Z     | OP_NEQQ    |
| Z      | vacío            | Z     | Z     | ninguna    |

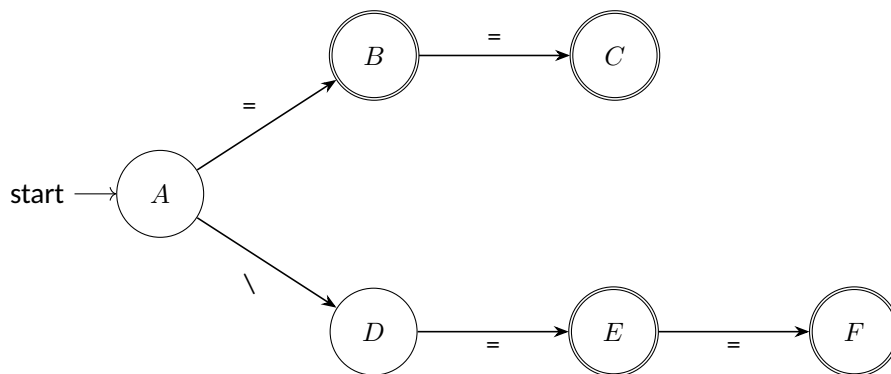


Figura 8: AFD obtenido por construcción de subconjuntos.

Para minimizar el AFD como clasificador de tokens, la partición inicial separa no aceptores y cada salida observable. Las firmas de A, D y Z son diferentes: conducen, respectivamente, a B, E y Z. El bloque se refina a  $\{A\}$ ,  $\{D\}$  y  $\{Z\}$ . La partición estable conserva siete estados y las cuatro salidas; ignorar las etiquetas permitiría fusiones incorrectas para un clasificador léxico. La Figura 9 muestra el AFD mínimo completo, con el sumidero Z al que llegan las transiciones de rechazo omitidas en la Figura 8.

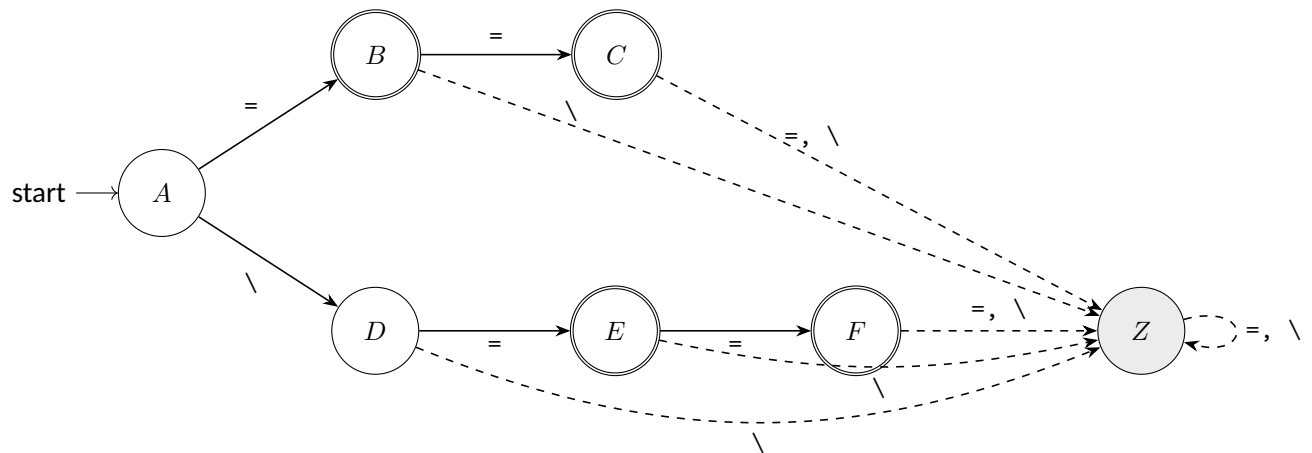


Figura 9: AFD mínimo etiquetado, con estado sumidero.

## Reglas de prioridad y máxima coincidencia

En cada posición se elige el candidato válido más largo y solo en empate se aplica la prioridad contractual.

| Caso                              | Regla aplicada                                                                        | Resultado esperado                     |
|-----------------------------------|---------------------------------------------------------------------------------------|----------------------------------------|
| \== / \= / == (P01)               | Se elige el lexema válido más largo (3 caracteres).                                   | OPERATOR \== de unificación            |
| =. ./ = (P02)                     | Más largo (3 caracteres) frente a prefijo de 1.                                       | OPERATOR =. de unificación             |
| == / = (P03)                      | Más largo (2 caracteres).                                                             | OPERATOR ==                            |
| : - / : (P05)                     | Operador exacto de dos caracteres; : solo es inválido.                                | OPERATOR : - de cláusula               |
| ? - / ? (P06)                     | Operador exacto de dos caracteres; ? solo es inválido.                                | OPERATOR ? - de consulta               |
| -> / - (P07)                      | Más largo (3 caracteres).                                                             | OPERATOR -> DCG                        |
| // / / (P08)                      | Más largo (2 caracteres).                                                             | OPERATOR // aritmético                 |
| ** / * (P09)                      | Más largo (2 caracteres).                                                             | OPERATOR ** aritmético                 |
| _ / _Variable (P10-P11)           | El guion solo es anónimo; con continuación gana la variable más larga.                | ANONYMOUS_VARIABLE o VARIABLE          |
| is / isla, mod / modulo (P12-P15) | Palabra reservada solo sin continuación; si hay continuación gana el átomo más largo. | OPERATOR u ATOM según frontera         |
| 1.25 (P16)                        | REAL más largo frente a prefijo INTEGER.                                              | REAL                                   |
| 12abc (P21)                       | Número con sufijo inválido: un único error hasta separador.                           | MALFORMED_NUMBER, sin tokens parciales |
| 5., .5, 1..2 (I07-I09)            | Detector numérico antes que DOT; no se emiten parciales.                              | Error, sin tokens parciales            |

Las palabras `is` y `mod` solo ganan cuando no tienen continuación de identificador; así, `is_` y `modulo` son átomos. El caso `12abc` consume un único fragmento numérico inválido y `x=y` produce tres tokens adyacentes porque el cursor avanza sin exigir separadores.

## Diseño e implementación

### Arquitectura de la solución

La arquitectura tiene cinco módulos con una sola dirección de dependencias: `tokens.py` define `TokenType`, `OperatorFamily`, el mapa único de operadores y el objeto `Token`; `errors.py` define el tipo de error y el diagnóstico; `symbol_table.py` implementa la tabla con deduplicación por par tipo y lexema; `lexer.py` expone el lexer y las funciones de lectura; y `main.py` ofrece la CLI.

El flujo recorre la entrada de izquierda a derecha con un cursor de índice, línea y columna: primero consume ignorables, luego literales, números, identificadores, operadores por longitud descendente, delimitadores y finalmente el carácter inválido. Cada token guarda la línea y columna de su primer carácter; la tabulación avanza una columna y CRLF cuenta como un único salto.

### Formato de salida

```
<TIPO_TOKEN, 'lexema', linea, columna>
```

Un fragmento real de la salida generada por el programa es:

```
<ATOM, 'padre', 2, 1>
<LPAREN, '(', 2, 6>
<ATOM, 'juan', 2, 7>
<COMMA, ',', 2, 11>
<ATOM, 'ana', 2, 13>
<RPAREN, ')', 2, 16>
<DOT, '.', 2, 17>
```

El archivo completo válido produce 207 tokens, 0 errores y 48 entradas de tabla. El programa con errores produce 69 tokens recuperados y 12 diagnósticos. Los comandos reproducibles son:

```
python -m compileall -q src tests tools
uv run --with pytest python -m pytest -q
python -m src.main tests/corpus/entradas/programa_valido.pl
python -m src.main tests/corpus/entradas/programa_con_errores.pl
```

### Tabla de lexemas

Se registran `ATOM`, `QUOTED_ATOM` y `VARIABLE` como identificadores, más `INTEGER`, `REAL` y `STRING` como literales conservables, siempre con el lexema original y sin conversiones. La clave de deduplicación es el par tipo y lexema. La variable anónima `_` nunca se registra porque cada aparición es independiente; operadores, delimitadores, espacios y comentarios tampoco.

## Tratamiento de errores léxicos

| Error                 | Ejemplo  | Mensaje producido    | Recuperación                                               |
|-----------------------|----------|----------------------|------------------------------------------------------------|
| Átomo sin cierre      | ' Juan   | átomo sin cierre     | descarta hasta antes del salto o hasta EOF                 |
| Cadena sin cierre     | " Juan   | cadena sin cierre    | descarta hasta antes del salto o hasta EOF                 |
| Comentario sin cierre | /* texto | bloque sin cierre    | consume hasta EOF                                          |
| Carácter no admitido  | @        | carácter no admitido | consume un carácter y continúa                             |
| Número mal formado    | 1 . . 2  | número mal formado   | consume la corrida completa de dígitos y puntos más sufijo |
| Escape inválido       | 'a\q'    | escape desconocido   | consume la pareja y sigue buscando el cierre               |

Los seis errores se acumulan sin detener el recorrido y cada diagnóstico incluye clase, fragmento, línea, columna y mensaje útil. Un escape desconocido registra el error, consume la pareja y continúa buscando el cierre. La salida real del archivo con errores contiene exactamente 12 diagnósticos, con tokens válidos después de cada error recuperable y ningún token después del bloque sin cierre final.

## Plan de pruebas y resultados

| ID  | Entrada o archivo               | Objetivo                   | Resultado esperado           | Resultado obtenido                           | Estado |
|-----|---------------------------------|----------------------------|------------------------------|----------------------------------------------|--------|
| V23 | padre(juan, ana) .              | Hecho y posiciones         | 7 tokens en columnas exactas | 7 tokens en columnas exactas                 | OK     |
| P01 | \==                             | Máxima coincidencia        | Operador completo            | Operador completo de unificación             | OK     |
| P21 | 12abc                           | Recuperación numérica      | Un error, sin parciales      | Un error, sin tokens parciales               | OK     |
| I02 | Átomo sin cierre + línea válida | Recuperación tras salto    | Error y tokens en línea 2    | Error y tokens en línea 2                    | OK     |
| F01 | programa_valido.pl              | Cobertura integral         | 207 tokens, 0 errores        | 207 tokens, 0 errores, 17 tipos y 6 familias | OK     |
| F02 | programa_con_errores.pl         | Diagnósticos y continuidad | 69 tokens y 12 errores       | 69 tokens, 12 errores y 28 entradas          | OK     |

La suite `tests/test_lexer.py` suma 85 pruebas: 3 del modelo Token, 26 válidas V01-V26, 22 de prioridad P01-P22, 16 inválidas I01-I16, 12 transversales T01-T12 y 6 de archivos F01-F06. La matriz completa queda disponible en `docs/informe/04_matriz_de_pruebas.md` y el corpus en `tests/corpus/`.

## Análisis de resultados

El archivo válido produce 207 tokens, 0 errores y 48 entradas, cubriendo los 17 tipos de token y las 6 familias. El archivo con errores produce 69 tokens, 12 errores de las 6 clases en 12 líneas distintas y 28 entradas, con tokens válidos después de cada error recuperable y ningún token después del bloque sin cierre final. El comando ejecutado fue `uv run --with pytest python -m pytest -q`, con resultado real de 85 aprobadas; además `python -m compileall src tests` terminó sin errores y `git diff --check` salió limpio.

La cobertura incluye cierres a EOF y antes de salto, escapes en ambos delimitadores, cinco formas numéricas mal formadas, caracteres aislados, CRLF, tabulación, comentarios dentro de literales, -12 como dos tokens y ) aislado como token. La dificultad principal fue distinguir el punto de fin de cláusula del punto decimal:

una cláusula que termina en número sin espacio (`mod 5.`) es número mal formado, por lo que los archivos válidos separan el número del punto final.

## Conclusiones

El trabajo muestra que un lexer riguroso nace de decisiones contractuales explícitas antes que del código: alfabeto ASCII, números sin signo, escapes cerrados por delimitador, comentarios no anidables y máxima coincidencia con prioridades resolvieron las ambigüedades sin casos especiales en la implementación.

Los objetivos se cumplieron de forma medible con 85 pruebas aprobadas, cobertura total de categorías y familias, y dos archivos completos que ejercitan el recorrido real. La separación entre especificación, implementación y validación permite rastrear cada regla desde su expresión regular hasta su prueba.

Las limitaciones son que el lexer solo reconoce el subconjunto contratado y que los archivos válidos deben separar con espacio un número del punto final por la regla de número mal formado. Como mejoras futuras se proponen modos de salida adicionales, un visualizador de autómatas generado desde las expresiones del código y pruebas de rendimiento sobre archivos grandes, sin ampliar el subconjunto ni entrar en análisis sintáctico.

## Referencias

- Lévano, M. (2026). *Tarea: análisis léxico del lenguaje Prolog. Investigación, análisis, diseño, implementación y validación* (INFO1148, sem2\_2026). Enunciado del curso.
- ISO/IEC. (1995). *ISO/IEC 13211-1:1995. Information technology—Programming languages—Prolog—Part 1: General core*.
- Python Software Foundation. (2026). *Regular expression operations (re)*. <https://docs.python.org/3/library/re.html>
- pytest-dev team. (2026). *pytest documentation*. <https://docs.pytest.org/>
- Astral Software. (2026). *uv documentation*. <https://docs.astral.sh/uv/>
- Equipo de la asignatura INFO1148. (2026). *Formato Informe Tarea*. Documento consultado en docs/referencias/Formato Informe Tarea.pdf.
- Equipo de trabajo. (2026). *Especificación léxica contractual, matriz de pruebas y código del analizador*. Repositorio <https://github.com/FernandoValdes01/Tarea01TC>.

## Anexos

| Recurso         | Enlace                                                                                                    |
|-----------------|-----------------------------------------------------------------------------------------------------------|
| Repositorio Git | <a href="https://github.com/FernandoValdes01/Tarea01TC">https://github.com/FernandoValdes01/Tarea01TC</a> |

### Anexo A. Evidencias adicionales

El contenido canónico del informe está en docs/informe/Fernandovaldestareaspark.tex; los nombres, fecha, autoría y resultados corresponden al trabajo de Fernando Valdés, Juan Muñoz y Vicente Rivera. La especificación contractual vive en docs/informe/01\_especificacion\_lexica.md y la matriz completa en docs/informe/04\_matriz\_de\_pruebas.md. El código está en src/ y el corpus en tests/corpus/.

Comandos ejecutados desde la raíz del repositorio:

```
python -m compileall -q src tests tools
uv run --with pytest python -m pytest -q
python -m src.main tests/corpus/entradas/programa_valido.pl
```

```
python -m src.main tests/corpus/entradas/programa_con_errores.pl  
python tools/build_report_diagrams.py  
git diff --check
```

Resultados reales: `compileall` sin errores, 85 pruebas aprobadas, PDF generado correctamente y `git diff --check` limpio. El archivo válido produce 207 tokens, 0 errores y 48 entradas; el archivo con errores produce 69 tokens, 12 errores y 28 entradas. Frente a la rúbrica: especificación y expresiones regulares en la sección 4, autómatas con AFN, subconjuntos y minimización en la sección 5, implementación con posiciones y tabla en la sección 7, y pruebas con errores léxicos en las secciones 8 y 9.

Las nueve figuras vectoriales del anexo se incorporan a continuación desde `automatas_appendix.pdf`, generadas con `build_report_diagrams.py` a partir de las fuentes de `docs/automatas/diagramas/`.

# Anexo A · guía de lectura

Secuencia de modelos que sostiene la especificación y la implementación del lexer.

## Qué se modela

Cada figura representa una decisión léxica concreta. Las cinco primeras describen categorías y elementos ignorables; la sexta integra el despacho por primera clase y máxima coincidencia; las dos últimas documentan la determinización y la minimización de un subconjunto de operadores.

## Convenciones de lectura

|            |                                                                        |
|------------|------------------------------------------------------------------------|
| ESTADO     | círculo con nombre ASCII estable, por ejemplo n0 o q2                  |
| ACEPTACIÓN | doble círculo; el rótulo inferior indica el token producido            |
| TRANSICIÓN | flecha etiquetada con el símbolo o clase consumida                     |
| BUCLE      | transición que vuelve al mismo estado; la punta toca la circunferencia |
| SUMIDERO   | estado gris para recorridos que no producen un token válido            |

### Ruta de lectura recomendada

- 01

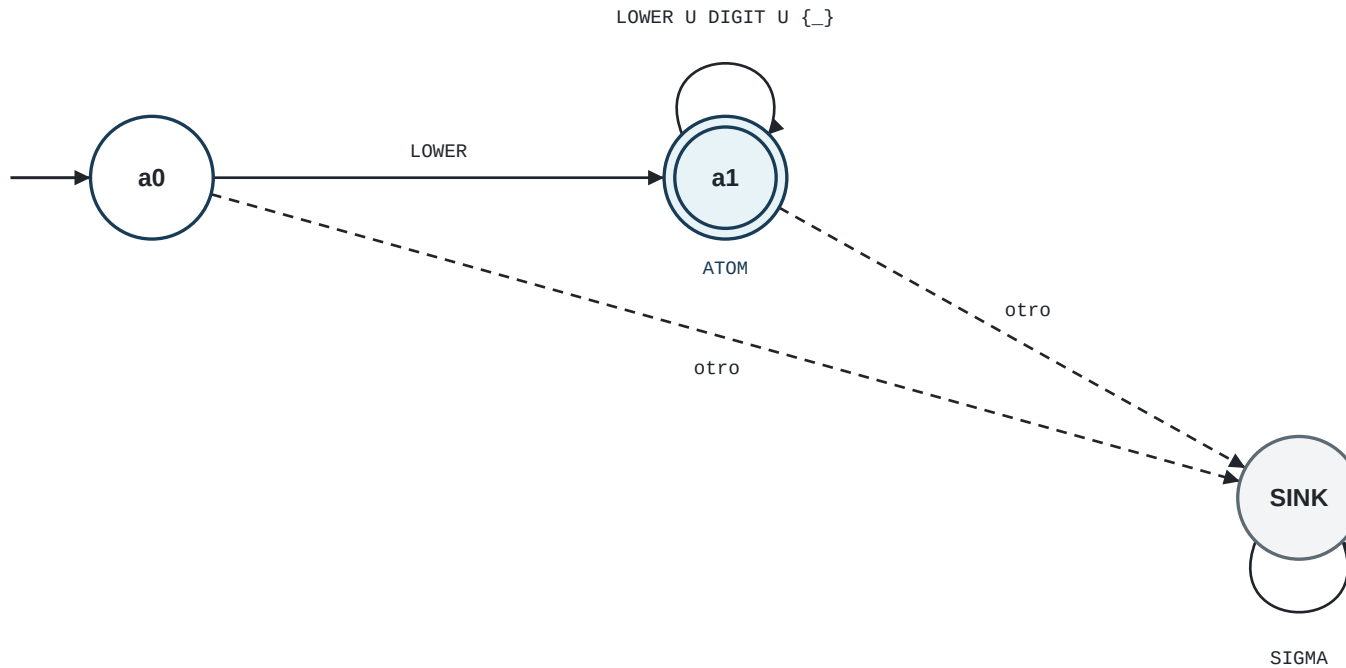
**Categorías**  
átomos, variables, números, literales y comentarios
- 02

**Operadores**  
prefijos comunes, máxima coincidencia y prioridades
- 03

**Construcción**  
AFN, subconjuntos, estado sumidero y AFD mínimo

# AFD para átomos no entrecomillados

Reconoce LOWER (LOWER U DIGIT U {\_})\* mediante máxima coincidencia.



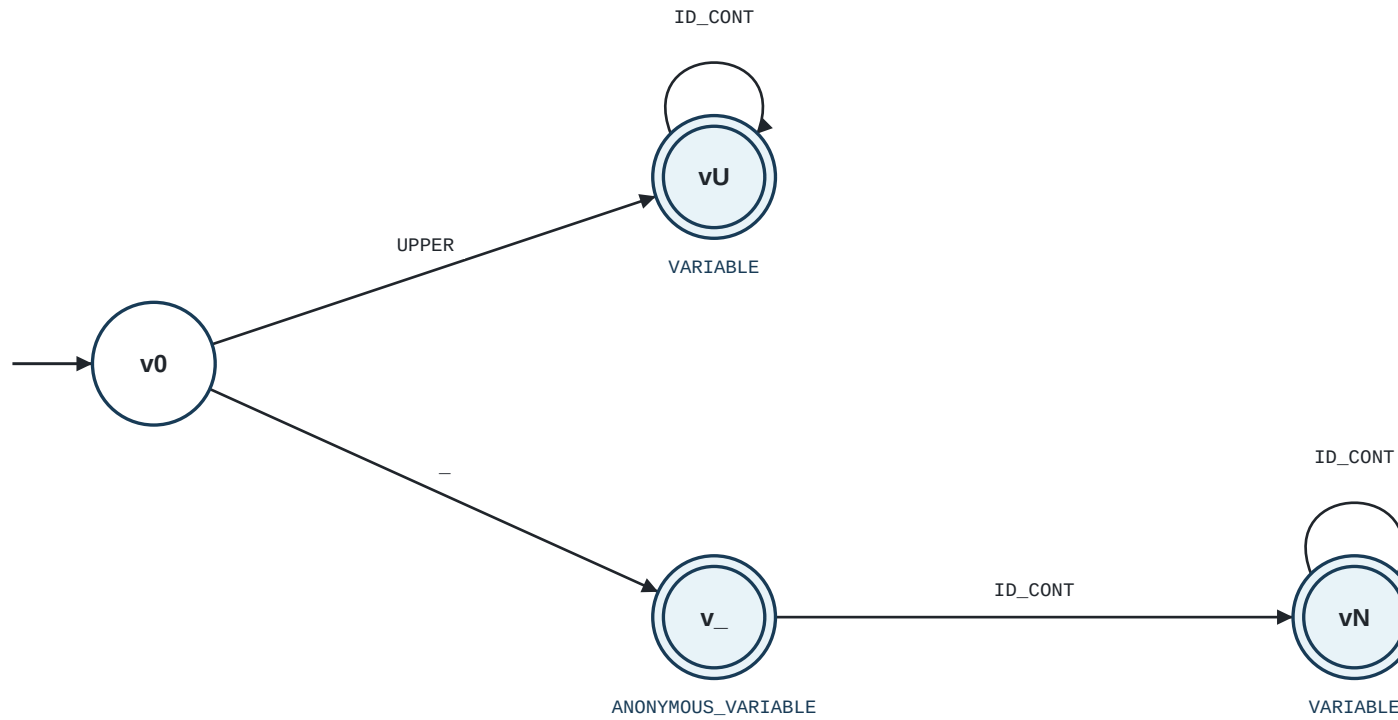
Estado aceptador y token emitido



Estado sumidero

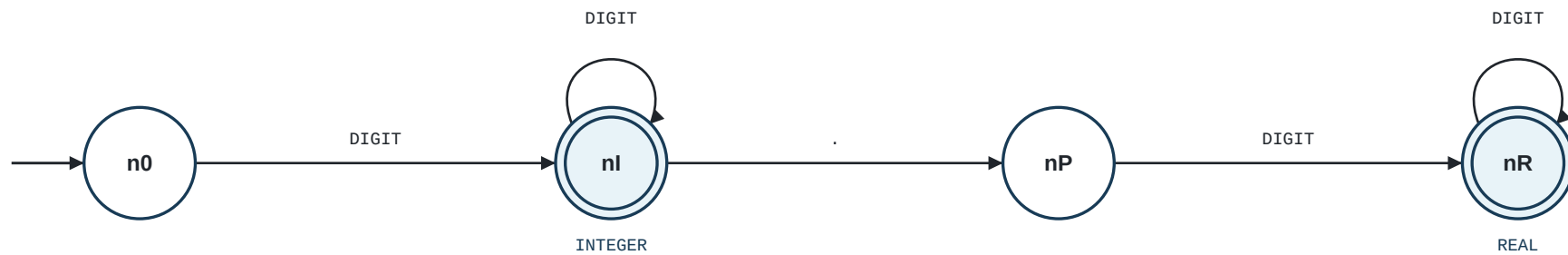
## AFD para variables

Distingue variables con nombre de la variable anónima `\_`.



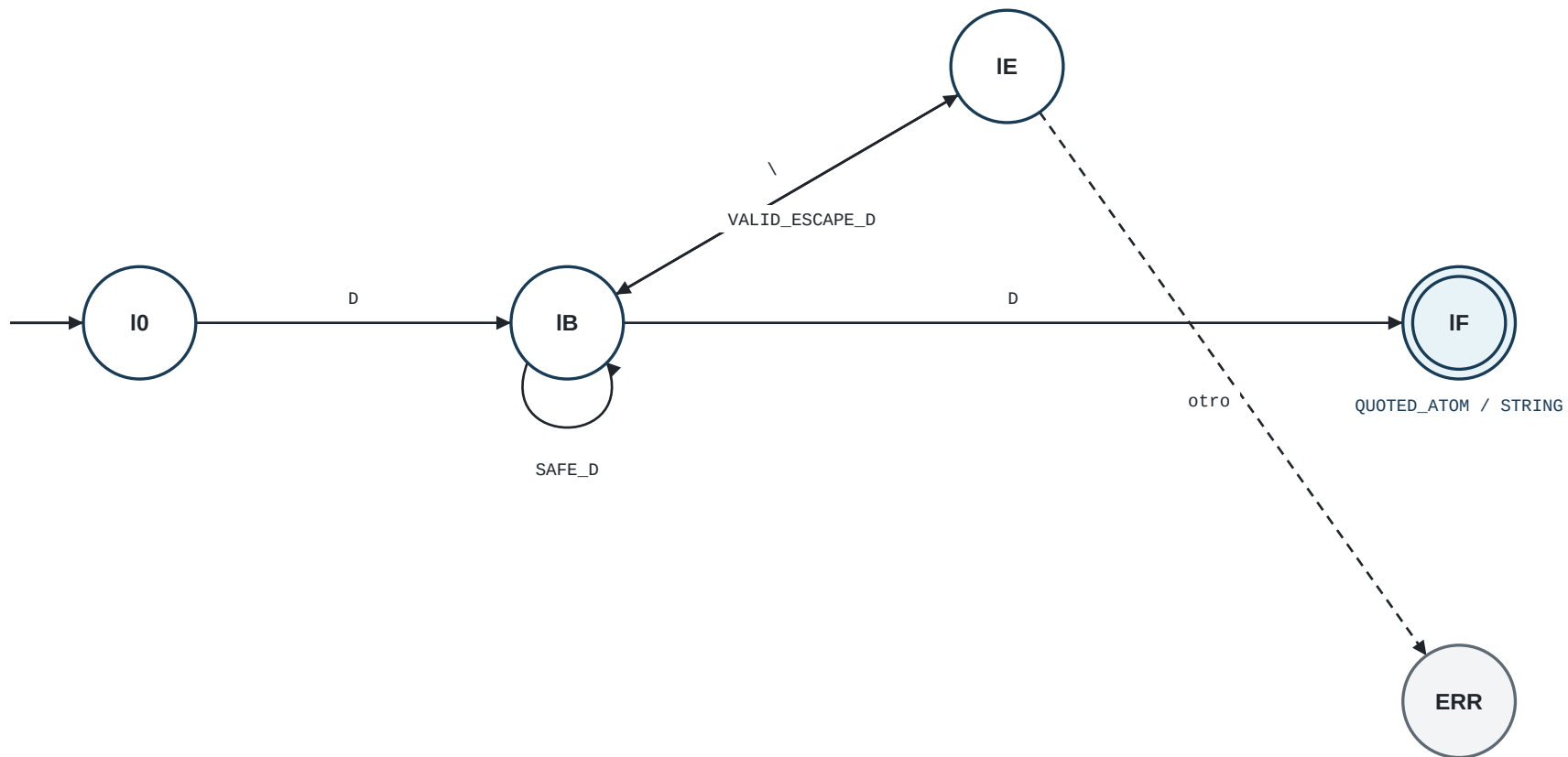
## AFD para números

Reconoce enteros y reales sin signo; el punto decimal exige un dígito posterior.



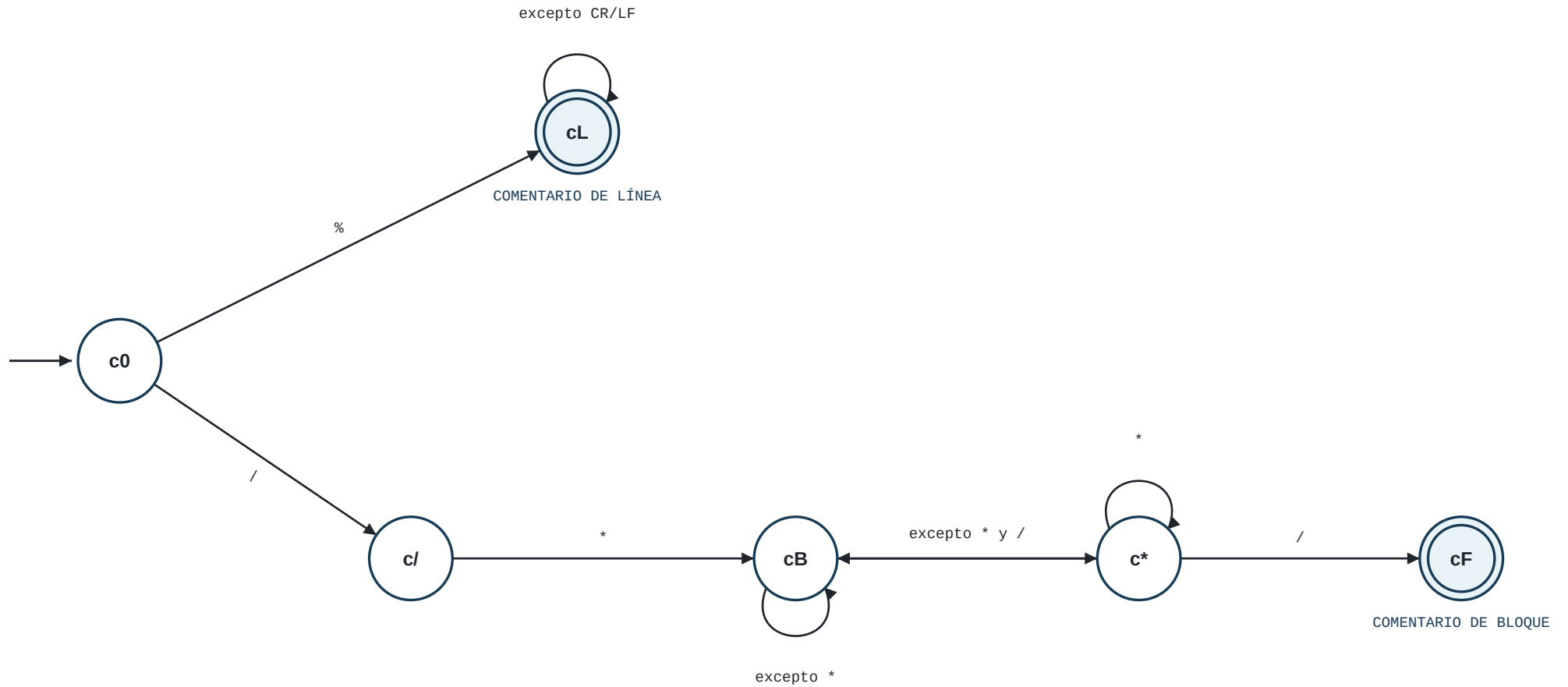
# AFD para literales entrecomillados

Esquema parametrizado por delimitador D: comilla simple o comilla doble.



## AFD para comentarios

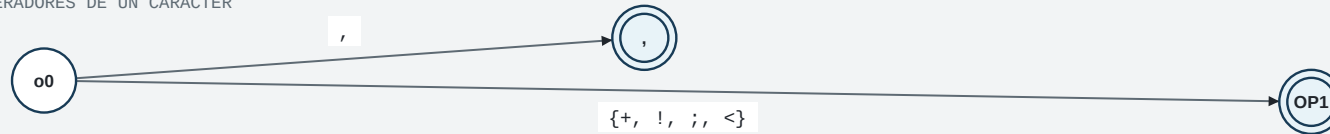
Ramas independientes para comentario de línea y comentario de bloque.



# Trie de operadores

Organización por prefijo para aplicar máxima coincidencia antes de emitir el token.

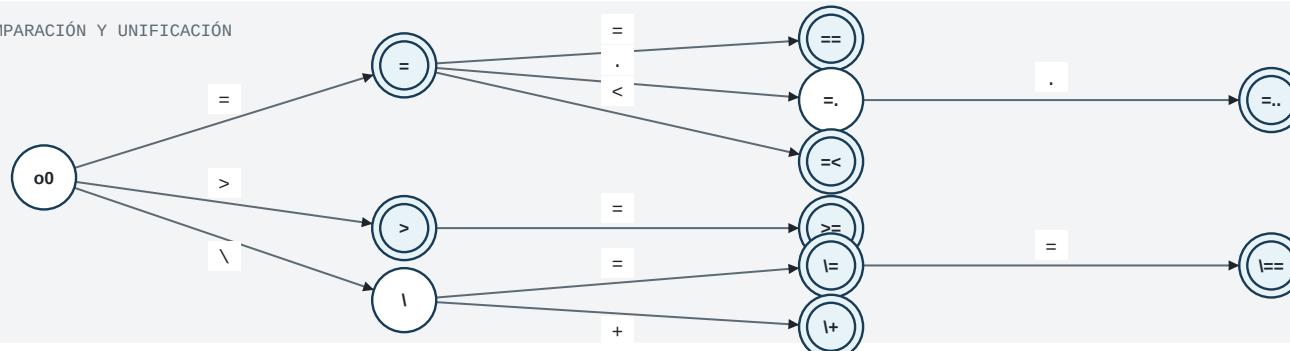
## OPERADORES DE UN CARÁCTER



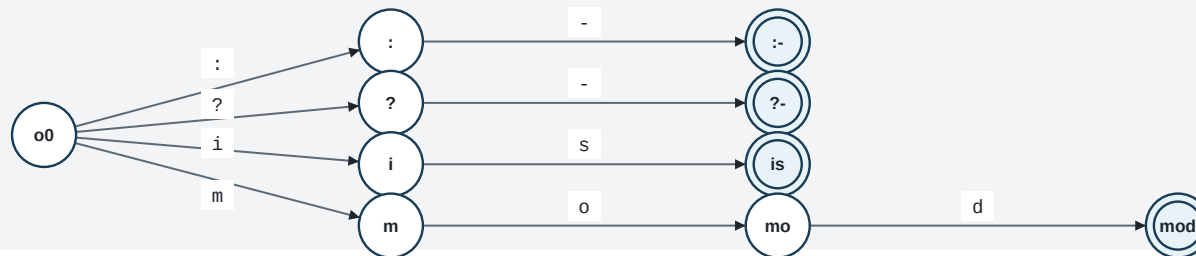
## PREFIJOS ARITMÉTICOS



## COMPARACIÓN Y UNIFICACIÓN



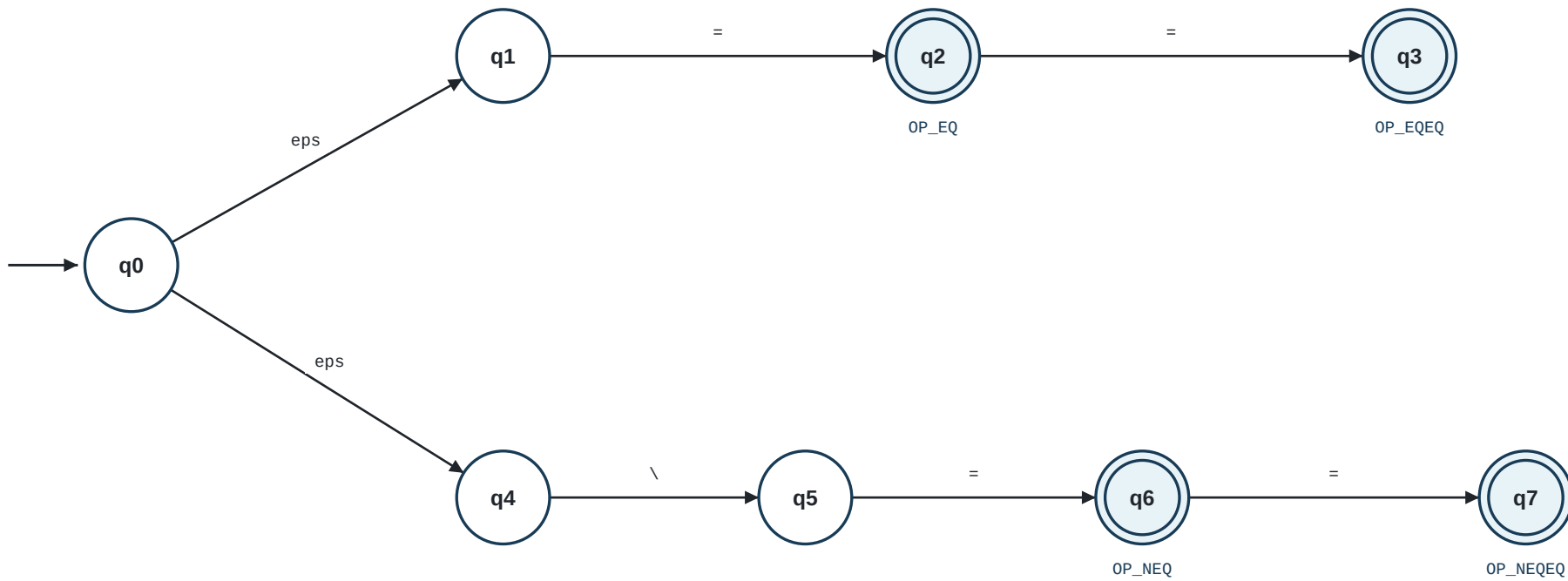
## CLÁUSULAS Y PALABRAS RESERVADAS



`is` y `mod` requieren frontera de palabra.

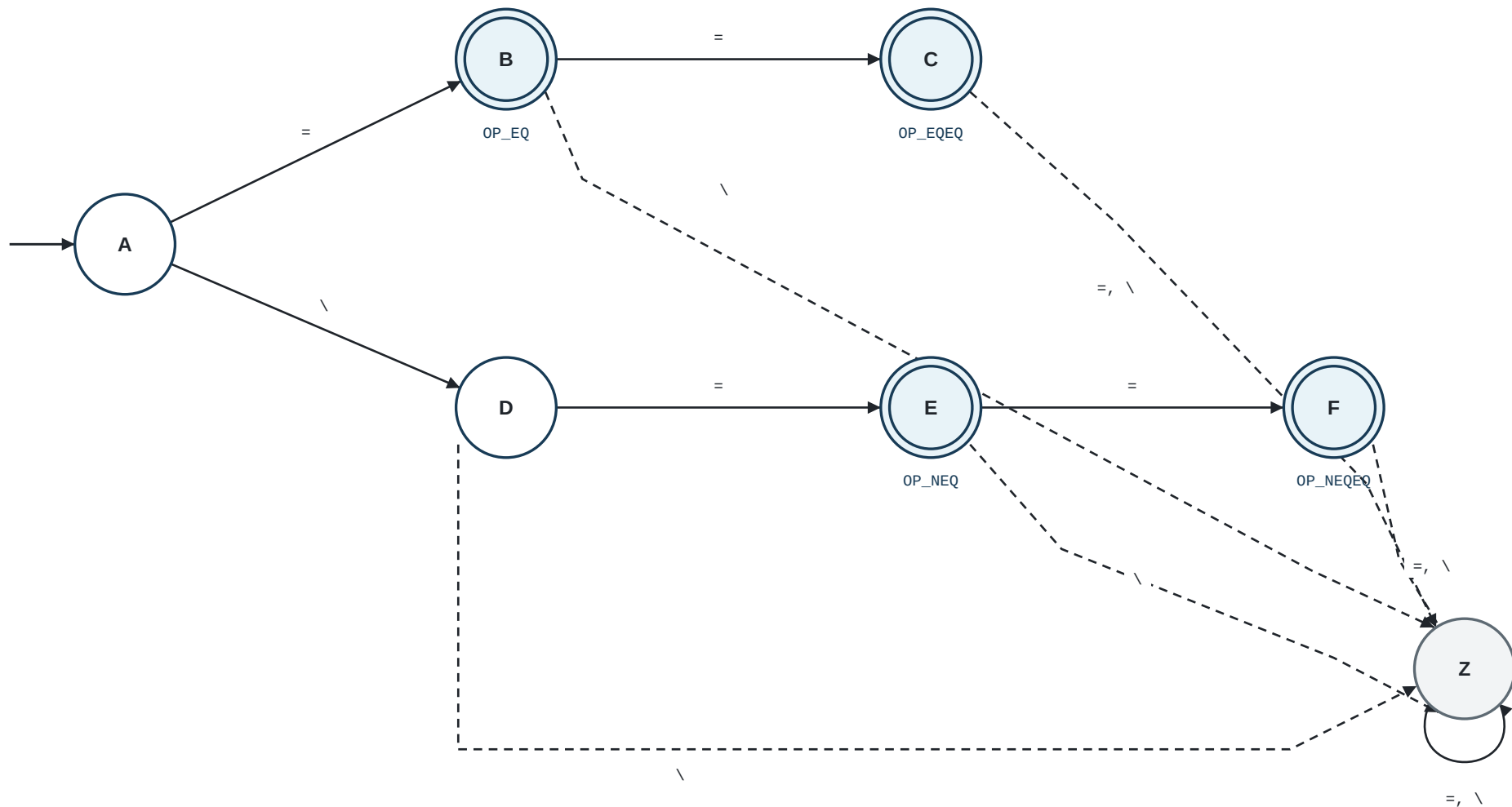
# AFN representativo de operadores

Lenguaje restringido a {=, ==, \=, \==} para mostrar la construcción formal.



# AFD obtenido por construcción de subconjuntos

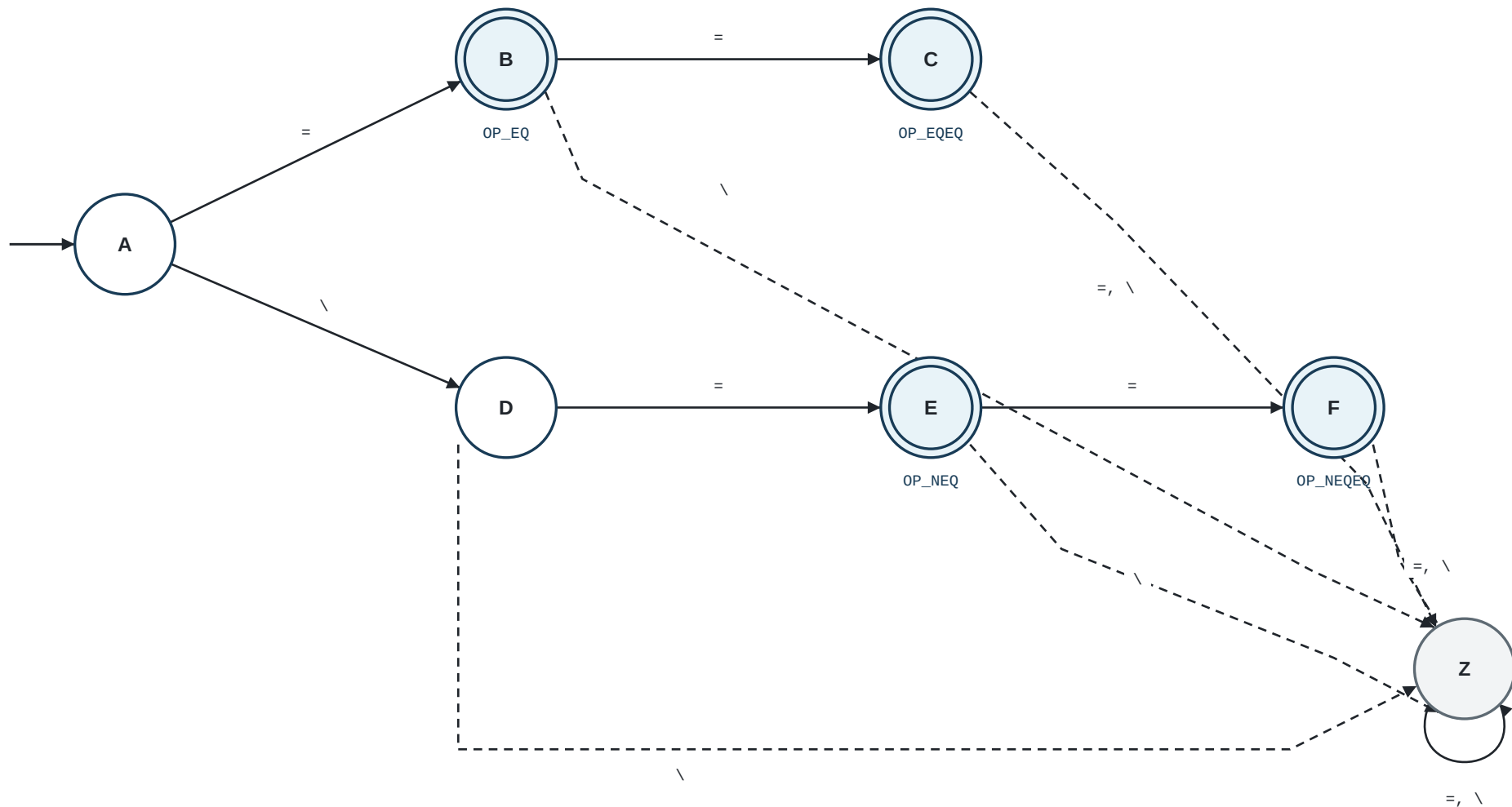
Cada estado representa un conjunto de estados del AFN; las salidas conservan la clasificación léxica.



Las líneas discontinuas llevan al sumidero Z; las rutas válidas permanecen en la zona superior.

## AFD mínimo etiquetado

La partición estable separa A, D y Z y conserva cuatro clases de aceptación observables.



Las líneas discontinuas llevan al sumidero Z; las rutas válidas permanecen en la zona superior.